TECHNICAL ARTICLE

Normalizing Flow Models: A Visual and Intuitive Guide

How invertible transformations turn simple Gaussian noise into complex, realistic data distributions

ME

Dr. Mohamed Elbasheer

Machine Learning Researcher

April 2026

ABSTRACT

Normalizing flows are a class of generative models that learn an invertible, differentiable mapping between a simple base distribution and a complex data distribution. Unlike GANs, they provide exact likelihood computation; unlike VAEs, the encoding is lossless and deterministic. This article introduces the mathematical foundations, the change-of-variables formula, efficient coupling-layer architectures, and a complete PyTorch implementation - all presented through visual intuition and a running example on the two-moons dataset.

1 Introduction

Generative modeling sits at the heart of modern machine learning: the ability to learn the underlying structure of data well enough to synthesize new, realistic samples. Three dominant paradigms have emerged - Generative Adversarial Networks (GANs), Variational Autoencoders (VAEs), and **Normalizing Flows**. Each makes different trade-offs between sample quality, training stability, and tractability of the likelihood.

Normalizing flows occupy a unique position: they are the only class that offers *exact* likelihood evaluation, stable training via maximum likelihood, and a fully invertible latent representation. This makes them especially attractive for scientific applications where probability estimates matter, anomaly detection, and uncertainty quantification.

Model	Exact Likelihood	Stable Training	Invertible Encoding
GAN	No	No (adversarial)	No
VAE	No (lower bound)	Yes	No (stochastic)
Normalizing Flow	**Yes**	**Yes**	**Yes**

2 Probability Background

A **probability density function (PDF)** $p(x)$ assigns a non-negative value to every point in the sample space such that $\int p(x)\,dx = 1$. The standard normal distribution $\mathcal{N}(0, I)$ is our reference distribution of choice because it is simple to sample from and its density is analytically tractable:

$$p_Z(\mathbf{z}) = \frac{1}{(2\pi)^{D/2}} \exp\left(-\frac{\|\mathbf{z}\|^2}{2}\right)$$

The central challenge is that real data - images, audio, molecular structures - follows a far more complex distribution $p_X(\mathbf{x})$ that cannot be written down in closed form. The question normalizing flows answer is: *can we learn a function that connects these two distributions?*

3 The Core Idea: Transforming Distributions

Suppose we have a simple random variable $\mathbf{z} \sim p_Z$ and we apply a smooth, invertible function f to obtain $\mathbf{x} = f(\mathbf{z})$. What is the density of $\mathbf{x}$?

Intuitively, probability acts like an incompressible fluid. If a transformation *squeezes* a region of space, the density in that region must increase to keep the total probability equal to 1. If it *stretches* space, the density decreases. The correction factor that tracks this volume change is the **Jacobian determinant**.

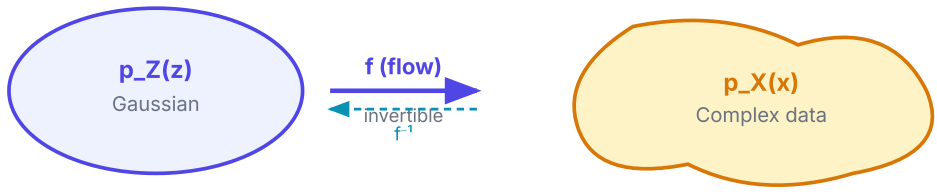

Figure 1. *A normalizing flow learns an invertible mapping f between a simple Gaussian base distribution and a complex data distribution. The inverse f^{-1} encodes data back to latent space.*

4 The Change of Variables Formula

The formal connection between p_Z and p_X is given by the **change of variables theorem**. For a diffeomorphism $f : \mathbb{R}^D \to \mathbb{R}^D$:

$$p_X(\mathbf{x}) = p_Z\big(f^{-1}(\mathbf{x})\big) \cdot \left|\det \frac{\partial f^{-1}}{\partial \mathbf{x}}\right|$$

Taking logarithms (for numerical stability):

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}) - \log |\det J_f(\mathbf{z})|$$

where J_f is the Jacobian matrix of f and $\mathbf{z} = f^{-1}(\mathbf{x})$. For a chain of K transformations $f = f_K \circ \cdots \circ f_1$:

$$\log p_X(\mathbf{x}) = \log p_Z(\mathbf{z}_0) + \sum_{k=1}^{K} \log \left|\det J_{f_k^{-1}}(\mathbf{z}_k)\right|$$

The Computational Challenge

Computing a general $D \times D$ determinant costs $\mathcal{O}(D^3)$. For a 64×64 image, $D = 4096$, making naive computation completely intractable. The key insight of practical flow architectures is to **design f so that its Jacobian is triangular**. The determinant of a triangular matrix is simply the product of its diagonal entries:

$$\det \begin{pmatrix} a_{11} & * & * \\ 0 & a_{22} & * \\ 0 & 0 & a_{33} \end{pmatrix} = a_{11} \cdot a_{22} \cdot a_{33} \quad \Rightarrow \quad \mathcal{O}(D) \text{ computation}$$

5 Affine Coupling Layers

The **affine coupling layer**, introduced in RealNVP (Dinh et al., 2016), is the most widely used building block. It achieves tractable Jacobians while allowing arbitrarily complex neural networks inside.

Forward Pass

Given input $\mathbf{z} \in \mathbb{R}^D$, split into two halves $\mathbf{z}_{1:d}$ and $\mathbf{z}_{d+1:D}$:

$$\mathbf{x}_{1:d} = \mathbf{z}_{1:d} \qquad \text{(identity)}$$
$$\mathbf{x}_{d+1:D} = \mathbf{z}_{d+1:D} \odot \exp\big(s(\mathbf{z}_{1:d})\big) + t(\mathbf{z}_{1:d})$$

where $s(\cdot)$ and $t(\cdot)$ are arbitrary neural networks outputting scale and shift vectors.

Inverse Pass

$$\mathbf{z}_{1:d} = \mathbf{x}_{1:d} \qquad \mathbf{z}_{d+1:D} = \big(\mathbf{x}_{d+1:D} - t(\mathbf{x}_{1:d})\big) \odot \exp\big(-s(\mathbf{x}_{1:d})\big)$$

Why the Jacobian is Triangular

Figure 2. *The Jacobian of an affine coupling layer is lower triangular. The top-left block is an identity matrix (first half is unchanged), and the bottom-right block is diagonal (elementwise scaling). The determinant reduces to a simple sum of log-scales.*

The log-determinant becomes:

$$\log|\det J| = \sum_{i=d+1}^{D} s_i(\mathbf{z}_{1:d})$$

> **KEY INSIGHT**
>
> The neural networks $s(\cdot)$ and $t(\cdot)$ can be *arbitrarily deep and wide* - their complexity does not affect the cost of computing the Jacobian determinant, because they only appear on the diagonal. This is the architectural trick that makes flows practical.

6 Training via Maximum Likelihood

Training a normalizing flow is straightforward: maximize the exact log-likelihood of the training data. Given a dataset $\{\mathbf{x}^{(i)}\}_{i=1}^{N}$:

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N}\left[\log p_Z\big(f_\theta^{-1}(\mathbf{x}^{(i)})\big) + \log\left|\det J_{f_\theta^{-1}}\big(\mathbf{x}^{(i)}\big)\right|\right]$$

This decomposes naturally into two interpretable terms:

1. **Latent fit:** the encoded points should resemble samples from the base Gaussian.
2. **Volume accounting:** the log-determinant penalizes transformations that collapse or over-expand probability mass.

Generating New Samples

Sampling from a trained flow is a single forward pass:

1. Sample $\mathbf{z} \sim \mathcal{N}(0, I)$ (trivially fast)
2. Compute $\mathbf{x} = f_\theta(\mathbf{z})$

7 Worked Example: Two-Moons Dataset

The two-moons dataset is a canonical 2D benchmark: two interleaved crescent-shaped clusters that no single Gaussian can capture. It demonstrates the need for multi-modal, non-convex density estimation.

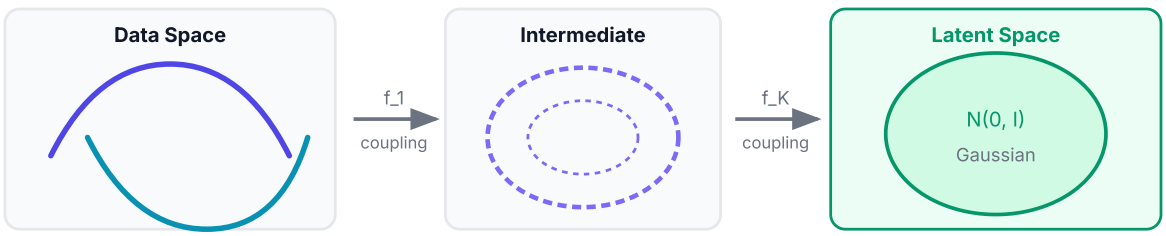

Figure 3. *A trained normalizing flow maps the complex two-moons distribution through a series of coupling layers until the encoded points resemble a Gaussian. Reversing the process generates new moon-shaped samples.*

8 PyTorch Implementation

The following is a minimal, complete implementation of a RealNVP-style flow trained on the two-moons dataset.

```
import torch, torch.nn as nn
from sklearn.datasets import make_moons

class CouplingLayer(nn.Module):
    def __init__(self, dim=2, hidden=128):
        super().__init__()
        d = dim // 2
        self.d = d
        # s and t are arbitrary MLPs
        self.s = nn.Sequential(nn.Linear(d,hidden), nn.Tanh(),
                               nn.Linear(hidden,hidden), nn.Tanh(),
                               nn.Linear(hidden, dim-d))
        self.t = nn.Sequential(nn.Linear(d,hidden), nn.Tanh(),
                               nn.Linear(hidden,hidden), nn.Tanh(),
                               nn.Linear(hidden, dim-d))

    def forward(self, z):          # z → x  (generate)
        z1, z2 = z[:,:self.d], z[:,self.d:]
        s, t = self.s(z1), self.t(z1)
        x2 = z2 * torch.exp(s) + t
        return torch.cat([z1, x2], 1), s.sum(1)   # (x, log_det)

    def inverse(self, x):          # x → z  (encode)
        x1, x2 = x[:,:self.d], x[:,self.d:]
        s, t = self.s(x1), self.t(x1)
        z2 = (x2 - t) * torch.exp(-s)
        return torch.cat([x1, z2], 1), -s.sum(1)


class NormalizingFlow(nn.Module):
    def __init__(self, dim=2, n_layers=8):
        super().__init__()
        self.layers = nn.ModuleList([CouplingLayer(dim) for _ in range(n_layers)])
        d = dim // 2
        # Alternate which half is transformed each layer
        self.perms = [torch.arange(dim) if i%2==0
                      else torch.cat([torch.arange(d,dim), torch.arange(d)])
                      for i in range(n_layers)]

    def log_prob(self, x):
        z, ldj = x, torch.zeros(len(x))
        for layer, p in zip(self.layers, self.perms):
            z = z[:, p]
            z, ld = layer.inverse(z)
            ldj += ld
        log_pz = -0.5*(z**2 + torch.log(2*torch.tensor(torch.pi))).sum(1)
        return log_pz + ldj

    @torch.no_grad()
    def sample(self, n):
        z = torch.randn(n, 2)
        for layer, p in zip(reversed(self.layers), reversed(self.perms)):
            z, _ = layer(z)
            z = z[:, torch.argsort(p)]
        return z


# --- Train ---
```

```
X, _ = make_moons(5000, noise=0.05)
data  = torch.tensor(X, dtype=torch.float32)
model = NormalizingFlow(dim=2, n_layers=8)
opt   = torch.optim.Adam(model.parameters(), lr=1e-3)

for epoch in range(2000):
    idx  = torch.randperm(len(data))[:512]
    loss = -model.log_prob(data[idx]).mean()
    opt.zero_grad(); loss.backward(); opt.step()
    if (epoch+1) % 500 == 0:
        print(f"Epoch {epoch+1}: loss = {loss.item():.4f}")
```

9 Notable Architectures

Architecture	Year	Key Contribution
NICE	2014	First additive coupling layers
RealNVP	2016	Affine coupling + multi-scale architecture
Glow	2018	Learned 1x1 convolutions for channel mixing
MAF / IAF	2017	Autoregressive flows (highly expressive)
Neural ODE Flow	2018	Continuous-time invertible dynamics
NSF	2019	Neural spline transformations

10 Applications

Scientific Computing

Molecular conformation generation, protein structure sampling, Bayesian inference in physics simulations.

Anomaly Detection

Assign exact probability scores to inputs; low-likelihood points are flagged as anomalies.

Image Generation

Glow achieves high-quality face synthesis with controllable latent interpolation.

Uncertainty Quantification

Lossless encoding enables principled credible intervals and calibrated predictions.

11 Summary

THREE CORE IDEAS

1. **Simple to complex:** a learned invertible function warps Gaussian noise into complex data without creating or destroying probability mass.
2. **Volume correction:** the Jacobian determinant accounts precisely for how space is stretched or compressed at each point.
3. **Triangular structure:** coupling layers design the Jacobian to be triangular, reducing an $\mathcal{O}(D^3)$ computation to $\mathcal{O}(D)$.

Normalizing flows remain the method of choice whenever exact likelihoods are required - in scientific simulation, uncertainty estimation, and density-based anomaly detection. While recent diffusion models surpass flows in raw image quality, the theoretical elegance and probabilistic rigor of flows make them indispensable in the generative modeling toolkit.

INTERACTIVE COMPANION

An interactive HTML guide with animated coupling layer diagrams, live distribution transformations, and a full runnable PyTorch demo accompanies this article. All concepts above can be explored visually without any setup.

Ref References

1. Dinh, L., Krueger, D., Bengio, Y. (2014). NICE: Non-linear Independent Components Estimation. *ICLR Workshop*.
2. Dinh, L., Sohl-Dickstein, J., Bengio, S. (2016). Density estimation using Real-valued Non-Volume Preserving transformations. *ICLR 2017*.
3. Kingma, D. P., Dhariwal, P. (2018). Glow: Generative Flow with Invertible 1x1 Convolutions. *NeurIPS 2018*.
4. Papamakarios, G., Nalisnick, E., et al. (2021). Normalizing Flows for Probabilistic Modeling and Inference. *JMLR*.
5. Rezende, D., Mohamed, S. (2015). Variational Inference with Normalizing Flows. *ICML 2015*.

#MachineLearning #GenerativeModels #NormalizingFlows #DeepLearning #PyTorch #DataScience #AI

Dr. Mohamed Elbasheer - April 2026

Open article.html in your browser and press "Save as PDF" to export.